



An Introduction to Recurrent Neural Networks for Beginners

A simple walkthrough of what RNNs are, how they work, and how to build one from scratch in Python.

JULY 24, 2019 | UPDATED JANUARY 13, 2026

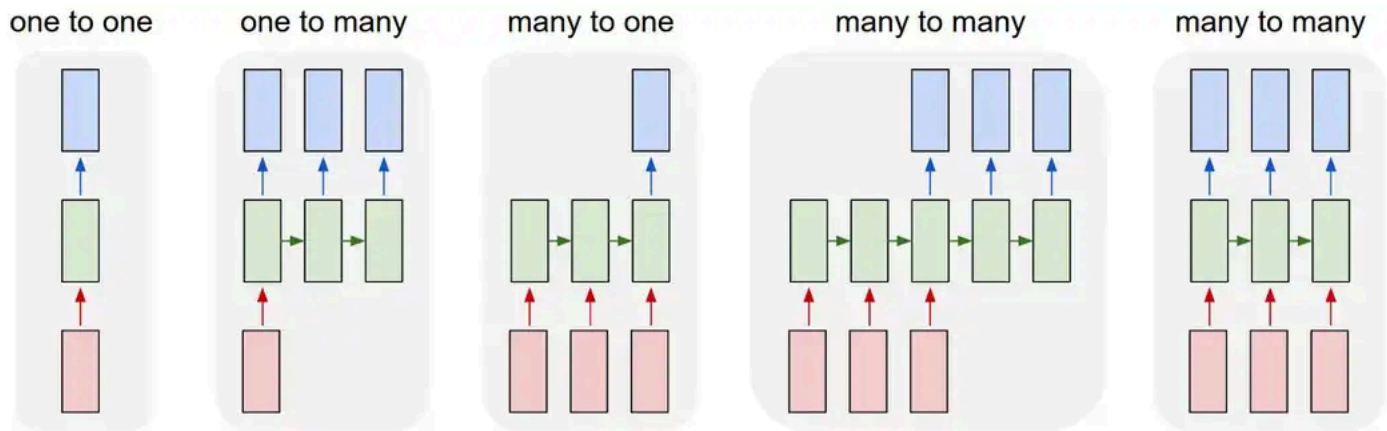
Recurrent Neural Networks (RNNs) are a kind of neural network that specialize in processing **sequences**. They're often used in [Natural Language Processing](#) (NLP) tasks because of their effectiveness in handling text. In this post, we'll **explore what RNNs are, understand how they work, and build a real one from scratch** (using only [numpy](#)) in Python.

This post assumes a basic knowledge of neural networks. My [introduction to Neural Networks](#) covers everything you'll need to know, so I'd recommend reading that first.

Let's get into it!

1. The Why

One issue with vanilla neural nets (and also [CNNs](#)) is that they only work with pre-determined sizes: they take **fixed-size inputs** and produce **fixed-size outputs**. RNNs are useful because they let us have **variable-length sequences** as both inputs and outputs. Here are a few examples of what RNNs can look like:



Inputs are red, the RNN itself is green, and outputs are blue. Source: [Andrej Karpathy](https://www.youtube.com/watch?v=K4C8Z0uXU08).

This ability to process sequences makes RNNs very useful. For example:

- **Machine Translation** (e.g. Google Translate) is done with “many to many” RNNs. The original text sequence is fed into an RNN, which then produces translated text as output.
- **Sentiment Analysis** (e.g. *Is this a positive or negative review?*) is often done with “many to one” RNNs. The text to be analyzed is fed into an RNN, which then produces a single output classification (e.g. *This is a positive review*).

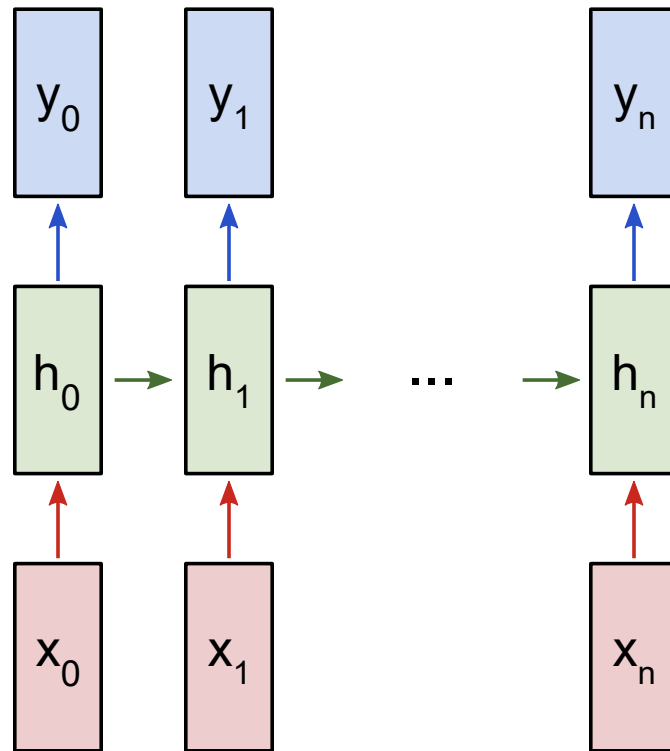
Later in this post, we’ll build a “many to one” RNN from scratch to perform basic Sentiment Analysis.

2. The How

Let’s consider a “many to many” RNN with inputs $x_0, x_1, \dots, x_n$ that wants to produce outputs $y_0, y_1, \dots, y_n$. These x_i and y_i are **vectors** and can have arbitrary dimensions.

RNNs work by iteratively updating a hidden state h , which is a vector that can also have arbitrary dimension. At any given step t ,

1. The next hidden state h_t is calculated using the previous hidden state h_{t-1} and the next input x_t .
2. The next output y_t is calculated using h_t .



Here's what makes a RNN *recurrent*: **it uses the same weights for each step**. More specifically, a typical vanilla RNN uses only 3 sets of weights to perform its calculations:

- W_{xh} , used for all $x_t \rightarrow h_t$ links.
- W_{hh} , used for all $h_{t-1} \rightarrow h_t$ links.
- W_{hy} , used for all $h_t \rightarrow y_t$ links.

We'll also use two biases for our RNN:

- b_h , added when calculating h_t .
- b_y , added when calculating y_t .

We'll represent the weights as *matrices* and the biases as *vectors*. These 3 weights and 2 biases make up the entire RNN!

Here are the equations that put everything together:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Don't skim over these equations. Stop and stare at this for a minute. Also, remember that the weights are *matrices* and the other variables are *vectors*.

All the weights are applied using matrix multiplication, and the biases are added to the resulting products. We then use [tanh](#) as an activation function for the first equation (but other activations like [sigmoid](#) can also be used).

No idea what an activation function is? Read my [introduction to Neural Networks](#) like I mentioned. Seriously.

3. The Problem

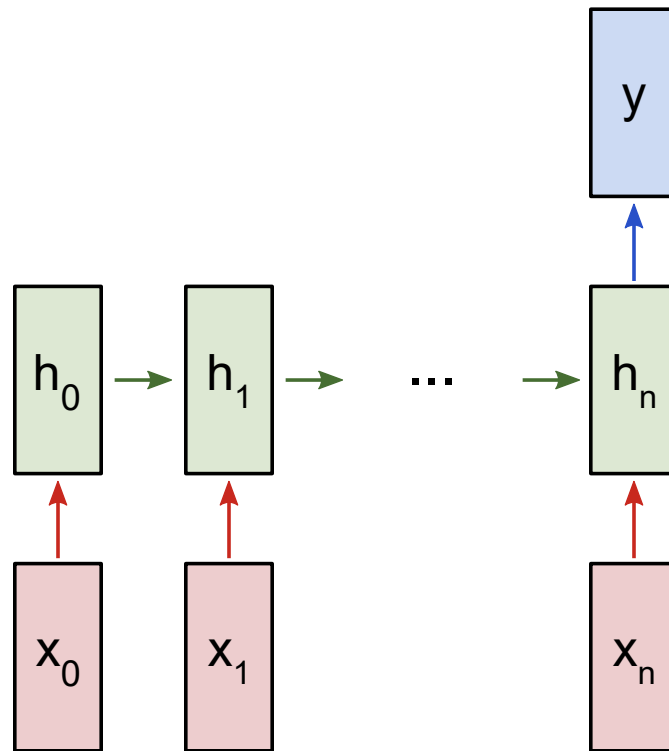
Let's get our hands dirty! We'll implement an RNN from scratch to perform a simple Sentiment Analysis task: **determining whether a given text string is positive or negative.**

Here are a few samples from the small [dataset](#) I put together for this post:

Text	Positive?
i am good	✓
i am bad	✗
this is very good	✓
this is not bad	✓
i am bad not good	✗
i am not at all happy	✗
this was good earlier	✓
i am not at all bad or sad right now	✓

4. The Plan

Since this is a classification problem, we'll use a "many to one" RNN. This is similar to the "many to many" RNN we discussed earlier, but it only uses the final hidden state to produce the one output y :



Each x_i will be a vector representing a word from the text. The output y will be a vector containing two numbers, one representing positive and the other negative. We'll apply [Softmax](#) to turn those values into probabilities and ultimately decide between positive / negative.

Let's start building our RNN!

5. The Pre-Processing

The [dataset](#) I mentioned earlier consists of two Python dictionaries:

```
# Header: data.py
train_data = {
    'good': True,
    'bad': False,
```

```
# ... more data
}

test_data = {
    'this is happy': True,
    'i am good': True,
    # ... more data
}
```

True = Positive, False = Negative

We'll have to do some pre-processing to get the data into a usable format. To start, we'll construct a **vocabulary** of all words that exist in our data:

```
# Header: main.py
from data import train_data, test_data

# Create the vocabulary.
vocab = list(set([w for text in train_data.keys() for w in text.split(' ')]))
vocab_size = len(vocab)
print('%d unique words found' % vocab_size) # 18 unique words found
```

vocab now holds a list of all words that appear in at least one training text. Next, we'll **assign an integer index** to represent each word in our vocab.

```
# Header: main.py
# Assign indices to each word.
word_to_idx = { w: i for i, w in enumerate(vocab) }
idx_to_word = { i: w for i, w in enumerate(vocab) }
print(word_to_idx['good']) # 16 (this may change)
print(idx_to_word[0]) # sad (this may change)
```

We can now represent any given word with its corresponding integer index! This is necessary because RNNs can't understand words - we have to give them numbers.

Finally, recall that each input x_i to our RNN is a *vector*. We'll use **one-hot vectors**, which contain all zeros except for a single one. The "one" in each one-hot vector will be **at the word's corresponding integer index**.

Since we have 18 unique words in our vocabulary, each x_i will be a 18-dimensional one-hot vector.

```
# Header: main.py
import numpy as np

def createInputs(text):
    """
    Returns an array of one-hot vectors representing the words
    in the input text string.
    - text is a string
    - Each one-hot vector has shape (vocab_size, 1)
    """
    inputs = []
    for w in text.split(' '):
        v = np.zeros((vocab_size, 1))
        v[word_to_idx[w]] = 1
        inputs.append(v)
    return inputs
```

We'll use `createInputs()` later to create vector inputs to pass in to our RNN.

6. The Forward Phase

It's time to start implementing our RNN! We'll start by initializing the 3 weights and 2 biases our RNN needs:

```
# Header: rnn.py
import numpy as np
from numpy.random import randn

class RNN:
    # A Vanilla Recurrent Neural Network.

    def __init__(self, input_size, output_size, hidden_size=64):
        # Weights
        self.Whh = randn(hidden_size, hidden_size) / 1000
        self.Wxh = randn(hidden_size, input_size) / 1000
        self.Why = randn(output_size, hidden_size) / 1000
```

```
# Biases
self.bh = np.zeros((hidden_size, 1))
self.by = np.zeros((output_size, 1))
```

Note: We're dividing by 1000 to reduce the initial variance of our weights. This is not the best way to initialize weights, but it's simple and works for this post.

We use [np.random.randn\(\)](#) to initialize our weights from the standard normal distribution.

Next, let's implement our RNN's forward pass. Remember these two equations we saw earlier?

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Here are those same equations put into code:

```
# Header: rnn.py
class RNN:
    # ...

    def forward(self, inputs):
        """
        Perform a forward pass of the RNN using the given inputs.
        Returns the final output and hidden state.
        - inputs is an array of one-hot vectors with shape (input_size, 1).
        """
        h = np.zeros((self.Whh.shape[0], 1))

        # Perform each step of the RNN
        for i, x in enumerate(inputs):
            h = np.tanh(self.Wxh @ x + self.Whh @ h + self.bh)

        # Compute the output
        y = self.Why @ h + self.by

        return y, h
```

Pretty simple, right? Note that we initialized h to the zero vector for the first step, since there's no previous h we can use at that point.

Let's try it out:

```
# Header: main.py
# ...

def softmax(xs):
    # Applies the Softmax Function to the input array.
    return np.exp(xs) / sum(np.exp(xs))

# Initialize our RNN!
rnn = RNN(vocab_size, 2)

inputs = createInputs('i am very good')
out, h = rnn.forward(inputs)
probs = softmax(out)
print(probs) # [[0.50000095], [0.49999905]]
```

If you need a refresher on Softmax, read my [quick explanation of Softmax](#).

Our RNN works, but it's not very useful yet. Let's change that...

7. The Backward Phase

In order to train our RNN, we first need a loss function. We'll use **cross-entropy loss**, which is often paired with Softmax. Here's how we calculate it:

$$L = -\ln(p_c)$$

where p_c is our RNN's predicted probability for the *correct* class (positive or negative). For example, if a positive text is predicted to be 90% positive by our RNN, the loss is:

$$L = -\ln(0.90) = 0.105$$

Want a longer explanation? Read the [Cross-Entropy Loss](#) section of my introduction to Convolutional Neural Networks (CNNs).

Now that we have a loss, we'll train our RNN using gradient descent to minimize loss. That means it's time to derive some gradients!

⚠ **The following section assumes a basic knowledge of multivariable calculus.** You can skip it if you want, but I recommend giving it a skim even if you don't understand much. **We'll incrementally write code as we derive results**, and even a surface-level understanding can be helpful.

If you want some extra background for this section, I recommend first reading the [Training a Neural Network](#) section of my introduction to Neural Networks. Also, all of the code for this post is on [Github](#), so you can follow along there if you'd like.

Ready? Here we go.

7.1 Definitions

First, some definitions:

- Let y represent the raw outputs from our RNN.
- Let p represent the final probabilities: $p = \text{softmax}(y)$.
- Let c refer to the true label of a certain text sample, a.k.a. the "correct" class.
- Let L be the cross-entropy loss: $L = -\ln(p_c)$.
- Let W_{xh} , W_{hh} , and W_{hy} be the 3 weight matrices in our RNN.
- Let b_h and b_y be the 2 bias vectors in our RNN.

7.2 Setup

Next, we need to edit our forward phase to cache some data for use in the backward phase. While we're at it, we'll also setup the skeleton for our backwards phase. Here's what that looks like:

```

# Header: rnn.py
class RNN:
    # ...

    def forward(self, inputs):
        """
        Perform a forward pass of the RNN using the given inputs.
        Returns the final output and hidden state.
        - inputs is an array of one-hot vectors with shape (input_size, 1).
        """
        h = np.zeros((self.Whh.shape[0], 1))

        self.last_inputs = inputs
        self.last_hs = { 0: h }

        # Perform each step of the RNN
        for i, x in enumerate(inputs):
            h = np.tanh(self.Wxh @ x + self.Whh @ h + self.bh)
            self.last_hs[i + 1] = h

        # Compute the output
        y = self.Why @ h + self.by

        return y, h

    def backprop(self, d_y, learn_rate=2e-2):
        """
        Perform a backward pass of the RNN.
        - d_y (dL/dy) has shape (output_size, 1).
        - learn_rate is a float.
        """
        pass

```

Curious about why we're doing this caching? Read my explanation in the [Training Overview](#) of my introduction to CNNs, in which we do the same thing.

7.3 Gradients

It's math time! We'll start by calculating $\frac{\partial L}{\partial y}$. We know:

$$L = -\ln(p_c) = -\ln(\text{softmax}(y_c))$$

I'll leave the actual derivation of $\frac{\partial L}{\partial y}$ using the Chain Rule as an exercise for you 😊, but the result comes out really nice:

$$\frac{\partial L}{\partial y_i} = \begin{cases} p_i & \text{if } i \neq c \\ p_i - 1 & \text{if } i = c \end{cases}$$

For example, if we have $p = [0.2, 0.2, 0.6]$ and the correct class is $c = 0$, then we'd get $\frac{\partial L}{\partial y} = [-0.8, 0.2, 0.6]$. This is also quite easy to turn into code:

```
# Header: main.py
# Loop over each training example
for x, y in train_data.items():
    inputs = createInputs(x)
    target = int(y)

    # Forward
    out, _ = rnn.forward(inputs)
    probs = softmax(out)

    # Build dL/dy
    d_L_d_y = probs
    d_L_d_y[target] -= 1

    # Backward
    rnn.backprop(d_L_d_y)
```

Nice. Next up, let's take a crack at gradients for W_{hy} and b_y , which are only used to turn the final hidden state into the RNN's output. We have:

$$\frac{\partial L}{\partial W_{hy}} = \frac{\partial L}{\partial y} * \frac{\partial y}{\partial W_{hy}}$$

$$y = W_{hy}h_n + b_y$$

where h_n is the final hidden state. Thus,

$$\frac{\partial y}{\partial W_{hy}} = h_n$$

$$\frac{\partial L}{\partial W_{hy}} = \boxed{\frac{\partial L}{\partial y} h_n}$$

Similarly,

$$\frac{\partial y}{\partial b_y} = 1$$

$$\frac{\partial L}{\partial b_y} = \boxed{\frac{\partial L}{\partial y}}$$

We can now start implementing `backprop()` !

```
# Header: rnn.py
class RNN:
    # ...

    def backprop(self, d_y, learn_rate=2e-2):
        """
        Perform a backward pass of the RNN.
        - d_y (dL/dy) has shape (output_size, 1).
        - learn_rate is a float.
        """
        n = len(self.last_inputs)

        # Calculate dL/dWhy and dL/dby.
        d_why = d_y @ self.last_hs[n].T
        d_by = d_y
```

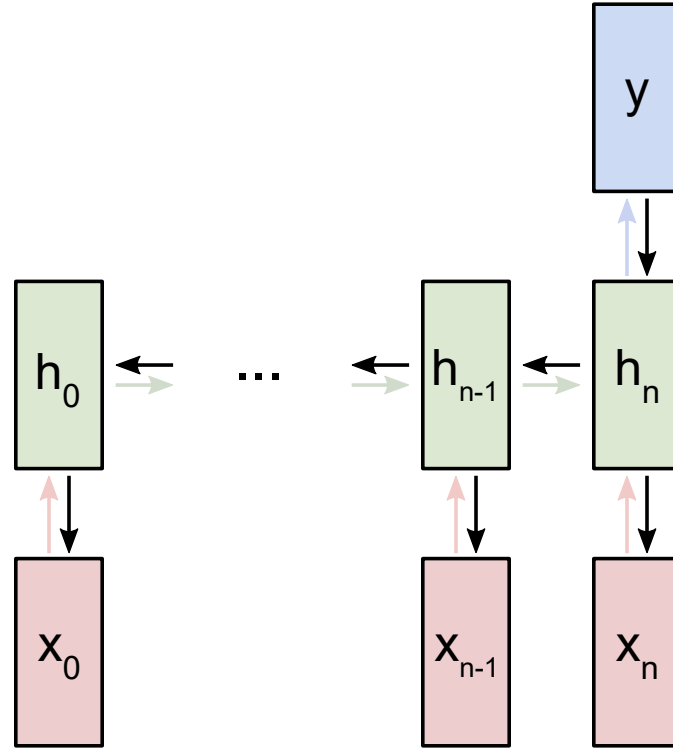
Reminder: We created `self.last_hs` in `forward()` earlier.

Finally, we need the gradients for W_{hh} , W_{xh} , and b_h , which are used *every* step during the RNN. We have:

$$\frac{\partial L}{\partial W_{xh}} = \frac{\partial L}{\partial y} \sum_t \frac{\partial y}{\partial h_t} * \frac{\partial h_t}{\partial W_{xh}}$$

because changing W_{xh} affects *every* h_t , which all affect y and ultimately L . In order to fully calculate the gradient of W_{xh} , we'll need to backpropagate through *all* timesteps,

which is known as **Backpropagation Through Time (BPTT)**:



W_{xh} is used for all $x_t \rightarrow h_t$ forward links, so we have to backpropagate back to each of those links.

Once we arrive at a given step t , we need to calculate $\frac{\partial h_t}{\partial W_{xh}}$:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

The derivative of $\tanh$ is well-known:

$$\frac{d \tanh(x)}{dx} = 1 - \tanh^2(x)$$

We use Chain Rule like usual:

$$\frac{\partial h_t}{\partial W_{xh}} = \boxed{(1 - h_t^2)x_t}$$

Similarly,

$$\frac{\partial h_t}{\partial W_{hh}} = \boxed{(1 - h_t^2)h_{t-1}}$$

$$\frac{\partial h_t}{\partial b_h} = \boxed{(1 - h_t^2)}$$

The last thing we need is $\frac{\partial y}{\partial h_t}$. We can calculate this recursively:

$$\begin{aligned}\frac{\partial y}{\partial h_t} &= \frac{\partial y}{\partial h_{t+1}} * \frac{\partial h_{t+1}}{\partial h_t} \\ &= \frac{\partial y}{\partial h_{t+1}} (1 - h_t^2) W_{hh}\end{aligned}$$

We'll implement BPTT starting from the last hidden state and working backwards, so we'll already have $\frac{\partial y}{\partial h_{t+1}}$ by the time we want to calculate $\frac{\partial y}{\partial h_t}$! The exception is the last hidden state, h_n :

$$\frac{\partial y}{\partial h_n} = W_{hy}$$

We now have everything we need to finally implement BPTT and finish `backprop()`:

```
# Header: rnn.py
class RNN:
    # ...

    def backprop(self, d_y, learn_rate=2e-2):
        """
        Perform a backward pass of the RNN.
        - d_y (dL/dy) has shape (output_size, 1).
        - learn_rate is a float.
        """
        n = len(self.last_inputs)

        # Calculate dL/dWhy and dL/dby.
        d_Why = d_y @ self.last_hs[n].T
        d_by = d_y

        # Initialize dL/dWhh, dL/dWxh, and dL/dbh to zero.
        d_Whh = np.zeros(self.Whh.shape)
        d_Wxh = np.zeros(self.Wxh.shape)
        d_bh = np.zeros(self.bh.shape)
```

```

# Calculate dL/dh for the last h.
d_h = self.Why.T @ d_y

# Backpropagate through time.
for t in reversed(range(n)):
    # An intermediate value: dL/dh * (1 - h^2)
    temp = ((1 - self.last_hs[t + 1] ** 2) * d_h)

    # dL/db = dL/dh * (1 - h^2)
    d_bh += temp

    # dL/dWhh = dL/dh * (1 - h^2) * h_{t-1}
    d_Whh += temp @ self.last_hs[t].T

    # dL/dWxh = dL/dh * (1 - h^2) * x
    d_Wxh += temp @ self.last_inputs[t].T

    # Next dL/dh = dL/dh * (1 - h^2) * Whh
    d_h = self.Whh @ temp

# Clip to prevent exploding gradients.
for d in [d_Wxh, d_Whh, d_Why, d_bh, d_by]:
    np.clip(d, -1, 1, out=d)

# Update weights and biases using gradient descent.
self.Whh -= learn_rate * d_Whh
self.Wxh -= learn_rate * d_Wxh
self.Why -= learn_rate * d_Why
self.bh -= learn_rate * d_bh
self.by -= learn_rate * d_by

```

A few things to note:

- We've merged $\frac{\partial L}{\partial y} * \frac{\partial y}{\partial h}$ into $\frac{\partial L}{\partial h}$ for convenience.
- We're constantly updating a `d_h` variable that holds the most recent $\frac{\partial L}{\partial h_{t+1}}$, which we need to calculate $\frac{\partial L}{\partial h_t}$.
- After finishing BPTT, we `np.clip()` gradient values that are below -1 or above 1. This helps mitigate the **exploding gradient** problem, which is when gradients become very large due to having lots of multiplied terms. [Exploding or vanishing gradients](#)

are quite problematic for vanilla RNNs - more complex RNNs like [LSTMs](#) are generally better-equipped to handle them.

- Once all gradients are calculated, we update weights and biases using **gradient descent**.

We've done it! Our RNN is complete.

8. The Culmination

It's finally the moment we been waiting for - let's test our RNN!

First, we'll write a helper function to process data with our RNN:

```
# Header: main.py
import random

def processData(data, backprop=True):
    """
    Returns the RNN's loss and accuracy for the given data.
    - data is a dictionary mapping text to True or False.
    - backprop determines if the backward phase should be run.
    """
    items = list(data.items())
    random.shuffle(items)

    loss = 0
    num_correct = 0

    for x, y in items:
        inputs = createInputs(x)
        target = int(y)

        # Forward
        out, _ = rnn.forward(inputs)
        probs = softmax(out)

        # Calculate loss / accuracy
        loss -= np.log(probs[target])
        num_correct += int(np.argmax
```